TorQ Ops x Guard: When the Support Agent Gets the Keys

How an AI agent gets real, hands-on access to a kdb+ operational stack - and still can't take it down.

A support engineer gets a page: some tickers in the consolidated book are showing null prices. There is no button for “find the broken feed” - the work is to poke around live state, compare sources, and follow the trail to the cause. This is the kind of first-line investigation we want to hand to an AI agent, which means giving it real, hands-on access to the kdb+ estate.

A fixed menu of tools cannot anticipate every support question, so a useful agent has to be able to try its own query. But an agent free enough to investigate is also free enough to do damage: on kdb+, one careless query can take the whole process down, and a wrong or manipulated action can reach production. Telling it to “be careful” is not a control.

This brief is for teams whose support already has some hands-on access to live processes - given a handle, told to be careful, left to learn as they go. (If your agents have no real access yet, you do not have this problem.) It is how we keep that access safe: the model can propose, but Guard decides. What follows is the approach we are proposing, and an honest account of where it holds and where it does not yet.

Guard in One Sentence

Guard is a deterministic checkpoint between an AI agent and the systems it can touch.

It works in three plain steps: the agent **proposes** an action; Guard **checks** it against a fixed policy and answers allow, ask-a-human, or block; and only an **approved** action is actually run. The agent never reaches the system directly. A block is not a dead end: Guard hands back the reason - which rule tripped, and what would be allowed instead - so the agent can revise its proposal and try again within the rules.

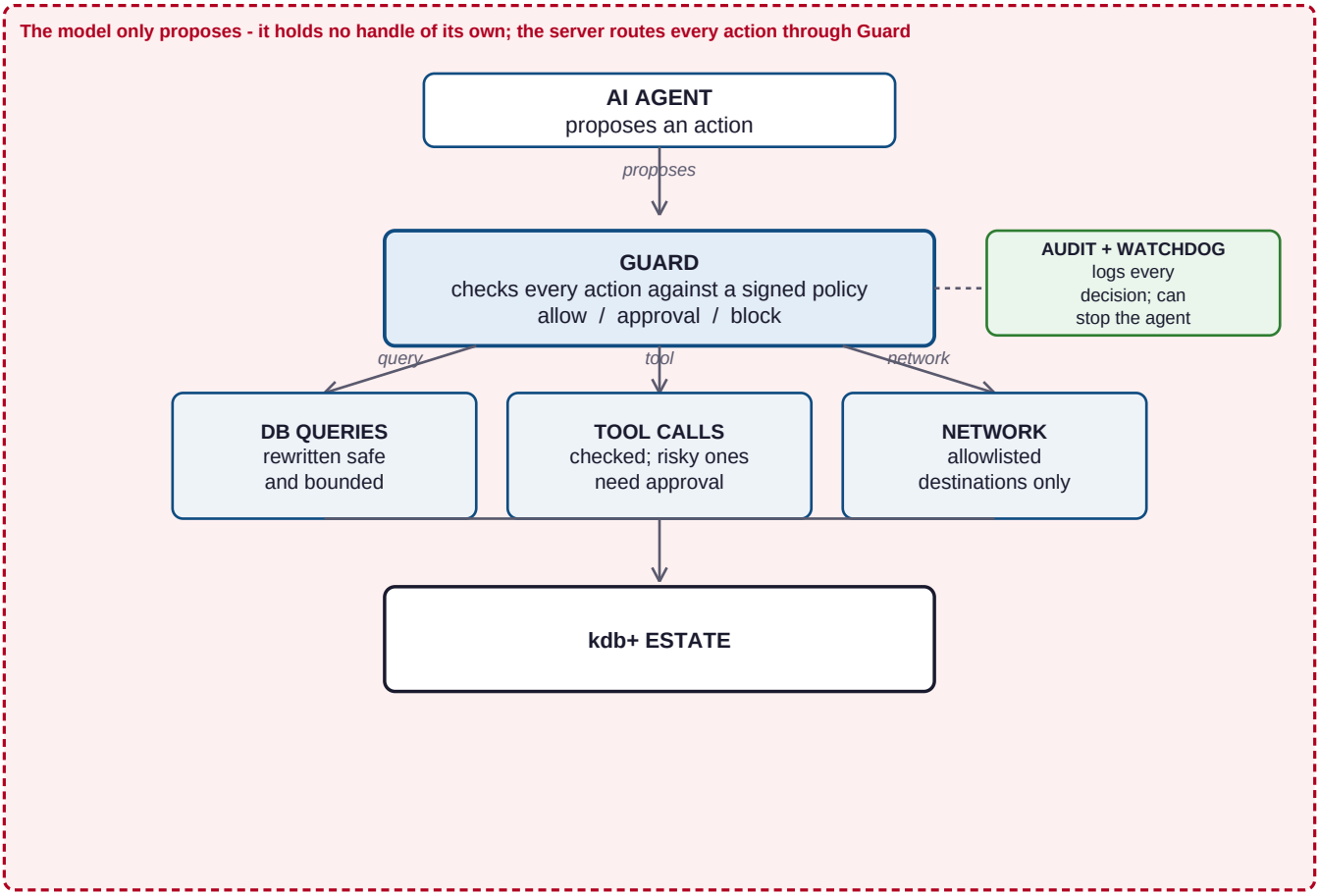
```
decision = guard.check(tool_name, tool_args, principal=user_id)
if decision.effect == "block":
    return refusal(decision)
if decision.effect == "require_approval":
    return approval_flow(decision)
return run_tool(tool_name, tool_args)
```

```
def evaluate(action):
    if policy_missing_or_invalid:
        return BLOCK
    findings  = enabled_rule_packs(action)
    findings += default_deny_grants(action)
    verdict   = most_severe(findings)   # block > approval > allow
    audit.record(action, verdict)
    supervisor.observe(action, verdict)
    return verdict
```

A prompt cannot negotiate with Guard. The same action, plus the same policy, always gives the same result.

Core Architecture

One decision point, three controlled exits: questions to the database, actions, and anything leaving the system.

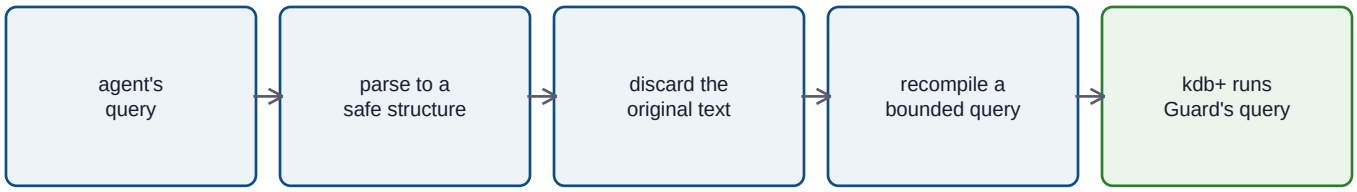


What each part is:

- **AI agent** - proposes an action; it has no authority to run anything itself.
- **Guard** - a signed-policy checkpoint that returns allow, approval or block; its decision runs in code the model never executes.
- **The three routes** - DB queries are rewritten into a bounded, allow-listed query (in the gate); tool calls are permission-checked, with risky ones held for approval (in the gate); and network egress is confined to allow-listed destinations by Guard's egress proxy where the operator deploys it - the platform layer covered under Deployment boundary below.
- **Audit + watchdog** - every decision is logged, and repeated refusals that keep failing the same way trip a breaker that halts the agent.
- **The boundary** - the model is given no database handle, shell or socket of its own; it only emits proposals, and the server runs every one through Guard first.

Query Regeneration

This is the core of Guard, and the part worth slowing down on. For database access Guard **does not run the model's query.** It reads what the query is asking for, throws the original text away, and writes a fresh, bounded query of its own.



The database receives Guard's query, never the model's original text.

A worked example. Back to that null-price page. Triaging the feed, the agent wants the average bid for NVDA and how many ticks came back null, and proposes this:

```
select avg bid, sum null bid by sym from prices_exchange where sym=`NVDA
```

Guard never sends that string. First it **lifts** the query into a plain structured request - data, not code:

```
{ "table":   "prices_exchange",
  "by":      ["sym"],
  "aggs":    [ {"fn": "avg", "col": "bid"},
               {"fn": "sum", "col": "bid", "of": "null", "as": "nb"} ],
  "filters": [ {"col": "sym", "op": "=", "value": "NVDA"} ] }
```

The raw text is gone - the backticks, the operators, anything executable. Then Guard **recompiles** that request into a brand-new query, stamping on a mandatory scan bound and row cap the agent never wrote. Guard knows **prices_exchange** is a real-time (RDB) table with no date partition, so it bounds the scan with a recent time-window rather than a date filter - and would reject a date filter against this table outright. This - and only this - is what kdb+ runs:

```
1000000 sublist (
  select avg bid, nb:sum null bid by sym
  from prices_exchange
  where time > .z.p - 30D00:00:00, sym=`NVDA )
```

The engineer still gets the number they were after - but the query is now bounded to a recent window and capped, so it cannot scan the whole estate or stall the process behind the live desk. (For a partitioned HDB table the same step stamps a date filter instead - Guard bounds each table the way its storage demands.)

The whole path is two calls - lift, then compile:

```
def safe_query(tool_input, principal):
    request = lift(tool_input["query"])           # parse -> structured request, or reject
    return compiler.compile(request, principal)   # rebuild a bounded, allow-listed query
```

Three stages, each closing a class of risk:

- **Parse** - pull out only the parts a query is allowed to have: table, columns, filters, time window. (The lift step above.)
- **Reject** - anything that doesn't fit that safe shape has nowhere to go. A delete, a shell call, a second statement hidden after a semicolon: none of them are in the grammar, so the request stops here, before any database sees it.
- **Compile** - write a new query from the structure, adding the row cap and the table's mandatory scan bound (a date filter for partitioned HDB tables, a recent time-window for real-time RDB ones), and honouring the access the caller is already entitled to. kdb+ runs Guard's text, never the model's.

So the dangerous cases never reach kdb+ - they fail at parse, or at compile:

```
delete from prices_exchange
   -> rejected at parse: only select / meta can be expressed

select sym from trade where date=...; system "..."
   -> rejected at parse: a second statement has no place in the structure

select pnl from trade where date=2026.06.23
   -> rejected at compile: column 'pnl' is not on the allow-list

select bid from prices_exchange where date=2026.06.23
   -> rejected at compile: prices_exchange is real-time; a date filter
      is not valid against it
```

The property that makes this strong: because Guard **emits** the query rather than approving the model's, a hijacked or simply mistaken model cannot get a dangerous query through - the dangerous form was never something Guard knows how to write.

What You Can Customise

Guard is policy-driven, so one engine is meant to adapt to very different agents. The policy lets you decide:

- **Tools** - which tools exist at all, which roles can call them, and which need a human to approve.
- **Data** - which tables, columns and rows are reachable, row caps, and the mandatory scan bound for each table: a date filter on partitioned (HDB) tables, a recent time-window on real-time (RDB) ones, set by the table's storage kind.
- **Network & process** - the platform boundary (allow-listed egress, read-only mounts, resource limits, no ambient shell or credentials) is the operator's to apply; Guard validates the deployment descriptor declares it.
- **Files** - which paths can be read or written, and which are off-limits.
- **Operations** - where a human must approve, spending ceilings, and the kill switch.
- **Rollout** - start small and widen. Begin with a deliberately narrow allow-list (default deny), run it in monitor (shadow) mode where Guard records the verdict it *would* have reached on real traffic without yet blocking, so you can measure false refusals before flipping to enforce. Legitimate queries that were blocked show up in the audit log, and a widen-from-log tool turns them into proposed policy additions for a human to sign off.
- **Audit** - how every decision is recorded, including hash-chained and off-host options.

The intent is that one engine covers a read-only reporting bot, a coding assistant locked to project files, an ops agent that needs sign-off for anything destructive, and an analyst limited to certain rows and date ranges - each just a different policy.

Why It Holds

- **Default deny:** if it isn't explicitly allowed, it doesn't happen.
- **Outside the model's reach:** the policy decision runs in code the model never executes - it only emits a proposal that Guard adjudicates.
- **No handle of its own:** the model is given no database connection, shell or socket; the server routes every action through Guard, so a careless or hijacked query is rewritten or refused before anything runs.
- **Signed policy:** the gate verifies the policy's signature against a pinned key before loading; a modified policy fails closed, so the agent cannot widen its own permissions.
- **Audit:** every decision is appended to a hash-chained log, so tampering is evident.
- **Watchdog:** repeated identical or escalating refusals - not ordinary revise-and-retry - trip a circuit breaker that halts the agent until an operator resets it.
- **Deployment boundary:** read-only mounts, resource limits and allow-listed egress are the operator's platform layer - Guard validates the deployment declares them, the platform enforces them.

What Guard Doesn't Do

Every deterministic gate makes a trade, and you should know its shape before you reach for one. Here is honestly where Guard's guarantee stops.

- **We trade a little reach for the guarantee.** The agent can only run what the allow-list permits. A legitimate query that falls outside the safe q subset - a custom function, an off-allow-list column, a shape the grammar doesn't model - is refused, not run. What you get back is a guarantee rather than a confident guess, but it is a real ceiling, and widening the grammar to cover more genuine diagnostic work is ongoing engineering, not a one-off.
- **Guard governs the query and the actions, not the whole world.** It makes the q the model writes safe and holds risky actions for approval. It is not, by itself, a defence against data leaving through some other channel the agent can reach, or against a compromised tool elsewhere in the loop. Guard is the kdb+-facing layer of a larger job, not the whole building.
- **Some of the boundary is the platform's, not the gate's.** Read-only mounts, CPU and memory limits, and network-egress allow-listing are enforced by the deployment - the container, the cluster, the egress proxy. Guard validates that the deployment declares them and ships the proxy, but it does not itself apply an OS-level limit. We are deliberate about which controls are gate code and which are the operator's platform layer, and we don't blur the two.
- **Approvals are only as good as the human reading them.** A gate that asks too often gets rubber-stamped. Guard leans on determinism precisely so it doesn't have to ask - it refuses outright, the same way every time - but the steps that do route to a person are only as strong as that person's attention.

Conclusion

Production AI does not fail safely by default. Once an agent has credentials, tools and network access, a bad output can become a real action.

Keeping agents in bounds means changing the execution model: restrict what the agent can reach, rebuild risky outputs into safe forms, require approval for sensitive steps, and record every decision. The model can stay flexible; the environment around it must be deterministic.

The practical shift: don't rely on the AI to know its limits. Build systems where the limits are enforced before anything runs.

So would we hand an AI agent real, hands-on access to a live kdb+ stack? Unguarded, no. Behind Guard - where the model only proposes, a deterministic gate disposes, and the only q that reaches kdb+ is one Guard wrote and bounded itself - that is a much shorter conversation with the risk team, and the one we are comfortable having.